CMPSC 448 Midterm Project

Michael Bostaph

10-11-2026

1 Problem Definition and Dataset Curation

1.1 Problem Definition

For this project, the students are asked to create two large language models (LLM) to determine if those two models can accurately identify which LLM (e.g. GPT, Claude, Gemini, etc.) generated a piece of text. The two LLMs that the students are supposed to create are convolutional neural network (CNN) and Recurrent Neural Network (RNN). The students then compare the performance of the two models to answer the two questions documented below.

The first question to be answered is, "Can we identify which LLM generated a response?" To answer this question, the students need to use the LLM output as their input into their models to identify which LLM generated it.

The Second question to be answered is, "Does the user's prompt help identify the LLM?" For this question, the students run three different experiments with three different inputs- user input only, LLM output only, and user input and LLM output - to classify which LLM generated a piece of text.

1.2 Dataset Curation

For this project, I decided to use the dataset, "lmsys/chatbot_arena_conversations," created by lmsys on the website, "huggingface.co." This dataset contains the following features: question_id, model_a, model_b, winner, judge, conversation_a, conversation_b, turn, anony, language, tstamp, openai_moderation, and toxic_chat_tag. For this project, only the model_a, and conversation_a features are used. The model_a feature simply contains the LLM that generated the piece of text found in conversation_a. The conversation_a feature contains a list of two dictionaries. The first dictionary contains the question the user asked the LLM. The second dictionary contains the answer the LLM provided.

2 CNN and RNN Implementation and How it is Trained

2.1 CNN Implementation and Training

For this project, the traditional CNN model is being repurposed from a image classifier to a text classifier. Thus, the Python class "TextCNN" is created to facilitate this implementation. The CNN model first tokenizes the text from conversation_a into data that the model can more easily understand. The class then applies 3 one-dimensional convolution filters before running the gelu function to add nonlinearity to each convolution. After that, the CNN model runs the max_pool1d function on the resultant feature vectors so that the strongest LLM detected by each filter is retained. Finally, the three pooled feature vectors

are concatenated as the model makes its prediction.

To train the CNN model, the model has up to 80% of the dataset available as training data. Each piece of data contains at most 256 tokens (characters) by default. Thus, shorter inputs will have to be extended to that limit, and longer inputs have to have their content truncated. For every batch of data, the model produces logits from the tokens and the attention mask from the convolution function. The model then calculates the cross-entropy loss against the model_a label before the model clears the gradients used in the convolution function. After that, the model backpropogates the loss and sets the maximum gradient magnitude to be 1.0. Finally, the model updates the CNN weights for future batches.

2.2 RNN implementation and Training

The basis for the RNN model is the bidirectional GRU text classifier (BiGRU). Like with the CNN model, this model predicts the model_a label using the input provided by conversation_a. This model is composed of three layers. The embeding layer takes the tokens and converts the tokens into 128 dimensional vectors. The padding token is specified so that padded positions described earlier are not learned by the model. The bidirectional GRU layer reads the tokens from the start of the text to the end of the text and vice versa. This allows the model to use the context produced on both sides of each token. Finally, the dropout layer concatenates the hidden state from each direction to produce a feature vector of size 384. Dropout is then applied before a linear filter converts the feature vectors into one logit for every possible model_a value.

For the training, the RNN model uses the same tokenized data splits and labels as the CNN model. However, this model uses separate parameters, forward passes, gradients, and evaluation results. For each training batch, the model moves the tokens, attention masks, and labels to the model. Next, the model runs the batch of data through the bidirectional GRU layer before computing the cross-entropy loss against the true model_a label. Then, the model clears the previous gradients, backpropogates the cross-entropy loss through the embedding layer, GRU layer, and dropout layer, and then sets the maximum gradient magnitude so that any unstable updates to the RNN weights don't happen.

3 The Presentation of the Results

The CNN and RNN models use the same validation set and metrics. But the way their individual predictions and metrics are calculated separately from one another. The results are printed in the terminal and in the same format for easier comparison. Each line contains the average cross-entropy loss on the validation data with lower values equating to better performance, the proportion of validation conversations for which the predicted model_a label is correct,

and the average F1 score across all model labels with every label being treated equally. After each line, the program will show selected examples of the test input, true label, predicted label, and the confidence level that the model has in its prediction. Using the flag "–show-examples" will allow the user to see more test data.

4 Analysis and Experiments

4.1 RQ1

For this question, the CNN model did a better job at identifying the correct LLM as seen in its higher F1 scores in each epoch. As seen in Figure 1, the CNN model is great at recognizing short patterns in wording, formatting, style, and repeated phrases. Since the program was run using only LLM output, the model can detect different response styles that are associated with different LLMs. However, it should be noted that the accuracy statistic shown in Figure 1 can be misleading as one model can achieve decent accuracy by choosing a model_a label that appears a lot in the dataset. The macro F1 scores are thus a better indication of accuracy as that statistic accounts for both uncommon and common LLMs. It should also be noted that the validation statistic in each model decreases with each epoch along with the F1 scores generally increasing in each epoch. This indicates that training data being supplied is sufficient for each model to improve its performance with each new batch.

```
PS C:\Users\mhbth\downloads> python Midterm_Project.py --rows 5000 --epochs 5 --input-mode assistant
CNN epoch 1: loss=2.1958 accuracy=0.2200 macro_f1=0.0768
CNN sample 1: input='It is difficult to pick a single "best" US president, as everyone has their own opinions and preferences about who they think was the best. However, some people may argue that
ington was the best US president for several reasons: 1. Strong leadership: George Washington was a strong leader...' expected='fastchat-t5-3b' predicted='vicuna-13b' confidence=0.2392
RNN epoch 1: loss=2.2021 accuracy=0.2120 macro_f1=0.0491
RNN sample 1: input='It is difficult to pick a single "best" US president, as everyone has their own opinions and preferences about who they think was the best. However, some people may argue that
ington was the best US president for several reasons: 1. Strong leadership: George Washington was a strong leader...' expected='fastchat-t5-3b' predicted='vicuna-13b' confidence=0.2108
CNN epoch 2: loss=2.1530 accuracy=0.2490 macro_f1=0.1123
CNN sample 1: input='It is difficult to pick a single "best" US president, as everyone has their own opinions and preferences about who they think was the best. However, some people may argue that
ington was the best US president for several reasons: 1. Strong leadership: George Washington was a strong leader...' expected='fastchat-t5-3b' predicted='vicuna-13b' confidence=0.2165
RNN epoch 2: loss=2.1815 accuracy=0.2010 macro_f1=0.0591
RNN sample 1: input='It is difficult to pick a single "best" US president, as everyone has their own opinions and preferences about who they think was the best. However, some people may argue that
ington was the best US president for several reasons: 1. Strong leadership: George Washington was a strong leader...' expected='fastchat-t5-3b' predicted='vicuna-13b' confidence=0.1998
CNN epoch 3: loss=2.1226 accuracy=0.2160 macro_f1=0.1078
CNN sample 1: input='It is difficult to pick a single "best" US president, as everyone has their own opinions and preferences about who they think was the best. However, some people may argue that
ington was the best US president for several reasons: 1. Strong leadership: George Washington was a strong leader...' expected='fastchat-t5-3b' predicted='vicuna-13b' confidence=0.2064
RNN epoch 3: loss=2.1613 accuracy=0.2130 macro_f1=0.0649
RNN sample 1: input='It is difficult to pick a single "best" US president, as everyone has their own opinions and preferences about who they think was the best. However, some people may argue that
ington was the best US president for several reasons: 1. Strong leadership: George Washington was a strong leader...' expected='fastchat-t5-3b' predicted='vicuna-13b' confidence=0.2237
CNN epoch 4: loss=2.0878 accuracy=0.2710 macro_f1=0.1595
CNN sample 1: input='It is difficult to pick a single "best" US president, as everyone has their own opinions and preferences about who they think was the best. However, some people may argue that
ington was the best US president for several reasons: 1. Strong leadership: George Washington was a strong leader...' expected='fastchat-t5-3b' predicted='vicuna-13b' confidence=0.2255
RNN epoch 4: loss=2.1419 accuracy=0.2220 macro_f1=0.1051
RNN sample 1: input='It is difficult to pick a single "best" US president, as everyone has their own opinions and preferences about who they think was the best. However, some people may argue that
ington was the best US president for several reasons: 1. Strong leadership: George Washington was a strong leader...' expected='fastchat-t5-3b' predicted='vicuna-13b' confidence=0.2236
CNN epoch 5: loss=2.0668 accuracy=0.2560 macro_f1=0.1344
CNN sample 1: input='It is difficult to pick a single "best" US president, as everyone has their own opinions and preferences about who they think was the best. However, some people may argue that
ington was the best US president for several reasons: 1. Strong leadership: George Washington was a strong leader...' expected='fastchat-t5-3b' predicted='vicuna-13b' confidence=0.3145
RNN epoch 5: loss=2.1208 accuracy=0.2250 macro_f1=0.1117
RNN sample 1: input='It is difficult to pick a single "best" US president, as everyone has their own opinions and preferences about who they think was the best. However, some people may argue that
ington was the best US president for several reasons: 1. Strong leadership: George Washington was a strong leader...' expected='fastchat-t5-3b' predicted='vicuna-13b' confidence=0.2830
```

Figure 1: CNN and RNN performance with LLM output as input

4.2 RQ2

with user input only, both models were not anywhere near as good at predicting LLM labels compared to LLM output only. As seen in Figure 2, the loss metric for both models never decreased substantially compared to LLM output only. This is largely due to user input naturally being more creative and free flowing when LLM outputs follow a specific pattern that can be detected by both humans and other LLMs. It should also be pointed out that the F1 scores for each model varies wildly due to pattern not being detected by the models.

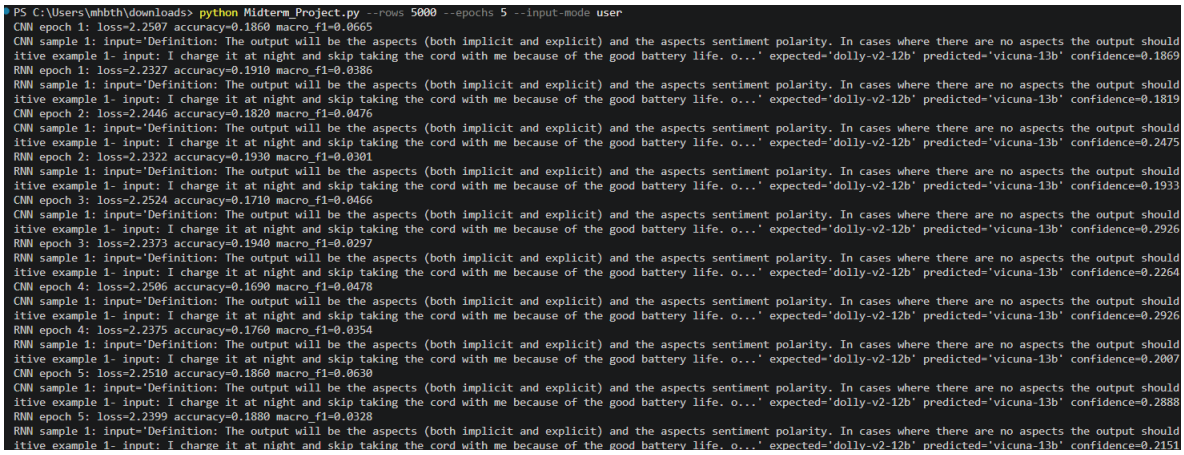


Figure 2: CNN and RNN performance with user input as input

With both user input and LLM output acting as inputs, the models both do better than having just user input, but not as well as only having LLM output. As seen in Figure 3, the loss metric for each model decreases, but still remains higher than the loss metrics as seen with only LLM output. The F1 scores increase for both models, but both end up not being nearly as high as seen in Figure 1. This is largely due to there only being a few patterns that can be detected by the models, with most of the patterns detected coming from the LLM output.

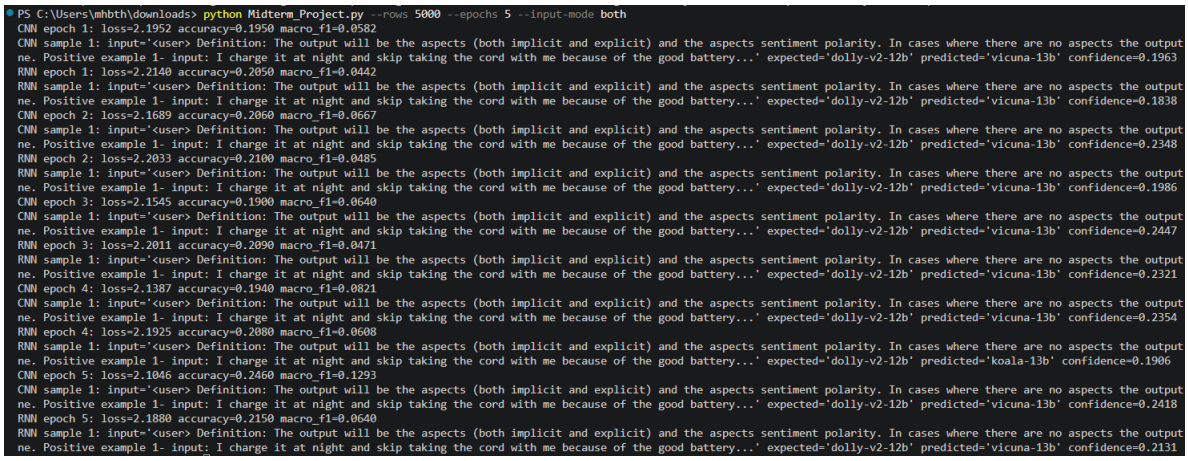


Figure 3: CNN and RNN performance with both user input and LLM output as input

5 Lessons and Experience Gained

From this project, I realized very early how much programming goes into developing just one of these models. Just being able to tokenize all of the characters took around 3 days to figure out. From there, other functionalities like developing the classes for each model would take around 1 to 2 days of coding. This is because of long-standing research and documentation as the CNN and RNN models have been and are a key focus in LLM research. Another lesson I took away was that Python has much better library support for developing LLMs compared to languages like Java and C. Python is designed such that anyone can upload a collection of functions or classes online and can be installed using the "pip install" command in the terminal of the user's ide. Python also is the choice amongst new programmers to learn how to write functions and classes. Thus, most people have stuck to using Python due to the language's wealth of libraries and vast community support not found with other programming languages. Finally, this project has helped me learned how to more efficiently document my code. With the Python file being around 313 lines of code, I didn't have the time to document every single line. Thus, with the assitance of Claude for proofreading, I was able to develop not only more understandable code, but also do it in a fraction of the time that I would use to take.